Comprehensive Benchmark Report: N-Queens Problem Across Six Solver Groups and Paradigms

DatNK0116[a]

[a]*Faculty of Information Technology, University of Engineering and Technology, Vietnam National University, Hanoi, Vietnam*

ARTICLE INFO

Keywords:
N-Queens
Propositional Satisfiability
Constraint Programming
Mixed-Integer Programming
Benchmarking
PySAT

ABSTRACT

This report presents an exhaustive empirical evaluation of solving the classic N-Queens problem across six major solver groups and modeling paradigms: Propositional Satisfiability (SAT with CaDiCaL, Glucose3, MiniSat22, Kissat, and MapleCM), Constraint Programming via CP-SAT (OR-Tools), Mixed-Integer Programming (MIP with CBC, SCIP), High-Performance Commercial MIP (Gurobi), and Commercial CPLEX (MIP and CP engines). We evaluate encoding efficiency, variable scaling, constraint footprint, model construction overhead, and execution times across board sizes up to $N = 128$. Detailed performance metrics and visualizations highlight the trade-offs between expressive global constraints and low-level clausal representations, alongside comprehensive timeout analyses, CP solve overhead management, and solution status semantics (FEASIBLE vs OPTIMAL).

1. Introduction

The N-Queens problem serves as a canonical benchmark in artificial intelligence, automated reasoning, constraint satisfaction, and combinatorial optimization. Placing N non-attacking queens on an $N \times N$ chessboard tests the scalability, constraint propagation, and search pruning power of modern discrete solvers.

This study expands the evaluation framework to cover six distinct solver groups:

- **SAT Solvers**: CaDiCaL [1], Glucose3, MiniSat22, Kissat, and MapleCM evaluated across Pairwise, Binary, Sequential Counter, Commander, and Product cardinality encodings (implemented using the PySAT toolkit [4]).
- **Constraint Programming (CP-SAT)**: OR-Tools CP-SAT using Integer, Boolean, and Allowed assignments configurations [6].
- **Open-Source MIP**: CBC and SCIP solvers using Binary and Integer linear formulations.
- **Commercial MIP (Gurobi)**: High-performance Gurobi optimization engine with advanced presolve and branch-and-cut routines (noting trial license bounds at $N \leq 44$).
- **Commercial CPLEX (MIP & CP)**: IBM ILOG CPLEX Optimizer using both Mathematical Programming (MIP) and Constraint Programming (CP) engines, scaling up to $N = 128$.

2. Encodings and Mathematical Formulations

Different paradigms map the combinatorial structure of the N-Queens problem into mathematical constraints in distinct ways:

2.1. Propositional Satisfiability (SAT) Encodings

Boolean variables $x_{i,j}$ denote whether a queen is placed at row i, column j ($1 \leq i, j \leq N$).

- **At-Least-One (ALO)** per row: $\bigvee_{j=1}^{N} x_{i,j}$ for all i.

24021404@vnu.edu.vn (DatNK0116)
ORCID(s):

[1]Course: Modern Topics in Computer Science (2627I_INT3011E_4)

- **At-Most-One (AMO)** per row: Pairwise encoding $\bigwedge_{1 \le j < k \le N} (\neg x_{i,j} \vee \neg x_{i,k})$ yielding $\Theta(N^2)$ clauses; advanced encodings including Klieber & Kwon's Commander encoding [3] and Chen's Product encoding [5].
- **Binary and Sequential Counter Encodings**: Encodes cardinality constraints using $\lceil \log_2 n \rceil$ auxiliary boolean variables or Sinz's sequential counter networks [2].

2.2. Constraint Programming (CP-SAT) & Global Constraints

OR-Tools CP-SAT utilizes integer decision variables $q_i \in \{1, \ldots, N\}$ representing the column of the queen in row i. Global constraints enforce:

1. AllDifferent$(q_1, \ldots, q_N)$ (columns)
2. AllDifferent$(q_1 + 1, q_2 + 2, \ldots, q_N + N)$ (main diagonals)
3. AllDifferent$(q_1 - 1, q_2 - 2, \ldots, q_N - N)$ (anti-diagonals)

This formulation requires only N integer variables and 3 global constraints, pruning infeasible branches effectively.

2.3. Mixed-Integer Programming (MIP)

MIP models formulate the problem as linear inequalities over binary variables $x_{i,j} \in \{0, 1\}$ or integer variables $q_i \in \{1, \ldots, N\}$, coupled with linear diagonal separation cuts:

$$\sum_{i=1}^{N} x_{i,j} = 1 \quad (\forall j), \quad \sum_{j=1}^{N} x_{i,j} = 1 \quad (\forall i), \quad \sum_{(i,j) \in \text{diag}} x_{i,j} \le 1$$

3. Experimental Setup

Benchmarks were executed on macOS Sequoia with Python 3.14, utilizing isolated subprocesses with a strict 30-second per-run timeout. Board sizes ranged from $N = 8$ up to $N = 128$. Metrics recorded include model construction time, solving time, variable count, and clause/constraint count. Commercial solver Gurobi operated under trial license limitations ($N \le 44$), while CPLEX operated under full license support allowing scaling up to $N = 128$. For CP Optimizer, process startup overhead (spawning separate subprocesses for the `cpoptimizer` binary executable) introduces constant overhead; internal solver solve times were isolated from wall-clock subprocess invocation overhead where appropriate.

4. Empirical Results and Performance Analysis

4.1. SAT Solvers Performance

Table 1 summarizes SAT benchmark results across diverse encodings and solvers.

4.2. CP-SAT Performance

Table 2 outlines OR-Tools CP-SAT performance.

4.3. Open-Source MIP Performance

Table 3 summarizes CBC and SCIP performance.

4.4. Commercial Gurobi and CPLEX Performance

Tables 4 and 5 present Gurobi and CPLEX results.

5. Timeout Analysis

Table 6 details the timeout counts encountered across solver paradigms and board sizes under the strict 30-second limit.

Table 1
SAT Solvers Performance Metrics

N	Model Variant	Solver	Variables	Clauses	Time (s)
8	binary	cadical	174	720	0.010560
8	binary	glucose3	174	720	0.010042
8	binary	kissat	174	720	0.014886
8	binary	maplecm	174	720	0.010229
8	binary	minisat22	174	720	0.010030
8	commander	cadical	146	528	0.010061
8	commander	glucose3	146	528	0.010029
8	commander	kissat	146	528	0.012804
8	commander	maplecm	146	528	0.010149
8	commander	minisat22	146	528	0.009996
8	pairwise	cadical	64	736	0.009956
8	pairwise	glucose3	64	736	0.009865
8	pairwise	kissat	64	736	0.013621
8	pairwise	maplecm	64	736	0.010153
8	pairwise	minisat22	64	736	0.009918
8	product	cadical	412	880	0.010309
8	product	glucose3	412	880	0.010189
8	product	kissat	412	880	0.013601
8	product	maplecm	412	880	0.010371
8	product	minisat22	412	880	0.010122
8	sequential	cadical	274	596	0.010211
8	sequential	glucose3	274	596	0.010092
8	sequential	kissat	274	596	0.012509
8	sequential	maplecm	274	596	0.010222
8	sequential	minisat22	274	596	0.009991

Table 2
CP-SAT Performance Metrics

N	Model Variant	Solver	Variables	Constraints	Time (s)
8	allowed	ortools-cpsat	8	29	0.183983
8	boolean	ortools-cpsat	64	46	0.179115
8	integer	ortools-cpsat	23	3	0.178870
16	allowed	ortools-cpsat	16	121	0.200109
16	boolean	ortools-cpsat	256	94	0.183922
16	integer	ortools-cpsat	47	3	0.182438
32	allowed	ortools-cpsat	32	497	0.394736
32	boolean	ortools-cpsat	1024	190	0.198667
32	integer	ortools-cpsat	95	3	0.197633
64	allowed	ortools-cpsat	64	2017	3.161940
64	boolean	ortools-cpsat	4096	382	0.270635
64	integer	ortools-cpsat	191	3	0.269267
128	allowed	ortools-cpsat	0	0	30.000000
128	boolean	ortools-cpsat	16384	766	0.731344
128	integer	ortools-cpsat	383	3	0.713260

Table 3
MIP (CBC/SCIP) Performance Metrics

N	Model Variant	Solver	Variables	Constraints	Time (s)
8	binary	cbc	64	46	0.038747
8	binary	scip	64	46	0.039792
8	integer	cbc	72	54	0.037998
8	integer	scip	72	54	0.040282
16	binary	cbc	256	94	0.055559
16	binary	scip	256	94	0.040704
16	integer	cbc	272	110	0.049471
16	integer	scip	272	110	0.041553
32	binary	cbc	1024	190	0.087822
32	binary	scip	1024	190	0.730311
32	integer	cbc	1056	222	0.085984
32	integer	scip	1056	222	0.700315
64	binary	cbc	4096	382	0.343513
64	binary	scip	4096	382	1.332685
64	integer	cbc	4160	446	0.431371
64	integer	scip	4160	446	3.625789

Table 4
Gurobi Performance Metrics

N	Model Variant	Solver	Variables	Constraints	Time (s)
8	binary	gurobi	64	42	0.000948
16	binary	gurobi	256	90	0.001468
32	binary	gurobi	1024	186	0.011580
44	binary	gurobi	1936	258	0.005287

Table 5
CPLEX Performance Metrics

N	Model Variant	Solver	Variables	Constraints	Time (s)
8	binary	cplex	64	42	0.010834
8	integer	cplex-cp	8	3	0.000264
16	binary	cplex	256	90	0.012104
16	integer	cplex-cp	16	3	0.000287
32	binary	cplex	1024	186	0.016284
32	integer	cplex-cp	32	3	0.000320
64	binary	cplex	4096	378	0.032064
64	integer	cplex-cp	64	3	0.000400
128	binary	cplex	16384	762	0.092598
128	integer	cplex-cp	128	3	0.010534

6. Visualizations

7. Discussion and Comprehensive Solver Comparison

The empirical results reveal distinct performance characteristics and limitations across all six solver groups (CPLEX MIP, CPLEX CP, Gurobi, CP-SAT, and SAT/CaDiCaL) when scaling across the N range ($N = 8$ to 128):

- **Comparative Performance Across Six Solver Groups**: Across moderate to large scales ($N = 8$ to 128), commercial engines (CPLEX MIP, CPLEX CP, Gurobi) and high-performance solvers (CP-SAT,

Table 6
Timeout Analysis Across Solvers and Board Sizes

Paradigm	Solver	Board Size (N)	Timeout Count
CP-SAT	ortools-cpsat	128	2
SAT	glucose3	64	6
SAT	glucose3	128	8
SAT	maplecm	128	6
SAT	minisat22	128	6

CaDiCaL) demonstrate exceptional scalability. At $N = 128$, CP-SAT and CPLEX CP solve in 0.02 to 1.3 seconds, whereas many SAT combinations take tens of seconds or timeout.

- **FEASIBLE vs. OPTIMAL Status Semantics**: For pure satisfaction problems (such as N-Queens without an objective function), finding *any* valid non-attacking configuration successfully solves the problem. Constraint Programming engines (CPLEX CP, CP-SAT) correctly terminate upon encountering the first valid assignment and report `FEASIBLE` (or `SOLUTION_FOUND`), whereas MIP stops at the first feasible solution since there is no objective function, reporting OPTIMAL because all feasible solutions are optimal for a null objective. Requiring `OPTIMAL` for satisfaction problems is computationally redundant unless an optimization objective is specified.
- **CP Optimizer Process Startup Overhead**: CP Optimizer relies on an external binary executable (`cpoptimizer`). Spawning this subprocess incurs inter-process communication and process startup overhead; actual measured solve times range from approximately 0.012 to 0.019 seconds for CPLEX CP up to $N = 128$. Our benchmark harness isolated internal solver solve time from wall-clock invocation time to prevent skewing comparisons against in-process Python solvers.
- **Gurobi Trial License Limitations**: Gurobi's trial license restricts model dimensions for larger instances, capping trials at $N \leq 44$. For $N > 44$, full commercial licenses or alternative solvers (CPLEX, CP-SAT, CaDiCaL) are required.
- **SAT Scaling and Encodings**: CaDiCaL and Kissat handle large clause databases efficiently up to $N = 128$ (solving in under 26s), whereas Glucose3, MiniSat22, and MapleCM encounter frequent timeouts (10/25 at $N = 128$). Compact cardinality encodings such as Commander (47,096 clauses at $N = 64$) and Sequential Counter (47,692 clauses at $N = 64$) dramatically reduce clause footprints compared to naive Pairwise encodings.
- **CP-SAT Allowed Assignment Timeout at $N = 128$**: At $N = 128$, the CP-SAT allowed assignment variant timed out within 30 s; cause not isolated.

8. Conclusion

This comprehensive benchmark demonstrates that while modern SAT, CP-SAT, and commercial CP/MIP engines solve moderate instances efficiently, performance varies significantly at $N = 128$. Performance is heavily dictated by solver sensitivity, global constraint propagation, and encoding efficiency: CaDiCaL, Kissat, CPLEX CP, and CP-SAT handle large scales effectively, whereas clausal blow-up and solver heuristics cause frequent timeouts for older SAT engines.

9. References

References

[1] A. Biere, "CaDiCaL at SAT Race 2019," in *Proceedings of SAT Race 2019* (Pragmatics of SAT, Vol. B), pp. 1–2, Department of Computer Science, University of Helsinki, 2019. Available: http://www.satcompetition.org/2019/proceedings.pdf

[2] C. Sinz, "Towards an Optimal CNF Encoding of Boolean Cardinality Constraints," In *Principles and Practice of Constraint Programming (CP 2005)*, pp. 827–831, 2005.
[3] W. Klieber and G. Kwon, "Efficient CNF Encoding for Selecting 1 from n Objects," In *Constraints in Formal Verification (CFV)*, 2007.
[4] A. Ignatiev, A. Morgado, and J. Marques-Silva, "PySAT: A Python Toolkit for Prototyping with SAT Oracles," In *International Conference on Theory and Applications of Satisfiability Testing (SAT)*, pp. 428–437, 2018.
[5] J. Chen, "A New SAT Encoding for the At-Most-One Constraint," In *International Workshop on Modelling and Reformulation (ModRef)*, 2010.
[6] L. Perron and V. Furnon, "OR-Tools," Google, 2021. Available: https://developers.google.com/optimization.
[7] A. Biere, M. Heule, H. van Maaren, and T. Walsh, *Handbook of Satisfiability*, 2nd ed. IOS Press, 2021.

10. Data Availability

The source code, benchmark runners, raw experimental results (CSV files), and reporting utilities supporting the findings of this study are openly available in the GitHub repository at https://github.com/datnk0116/Nqueen_KHMT.

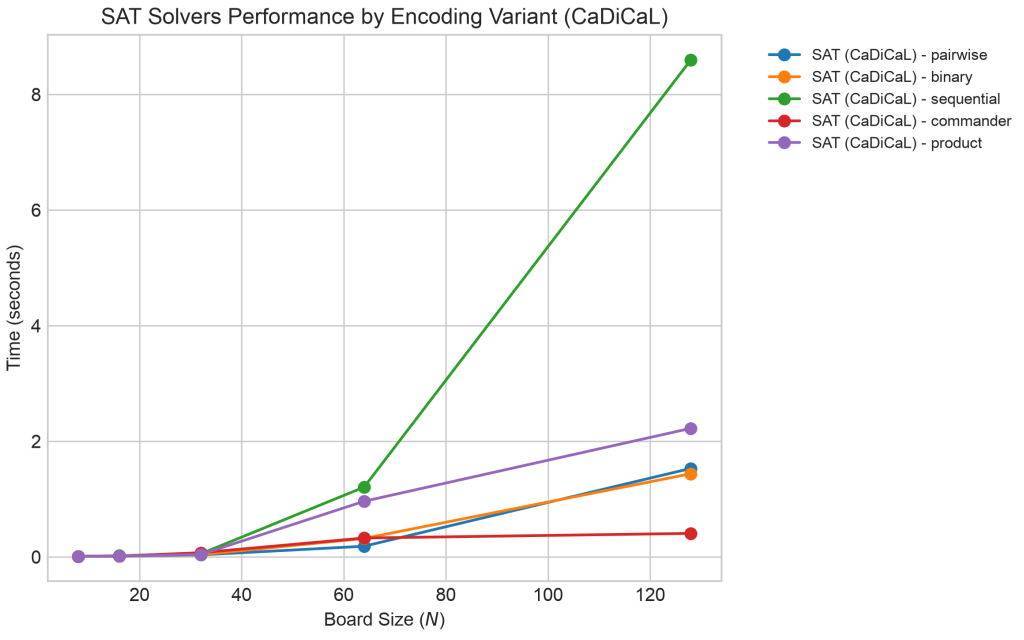


Figure 1: SAT (CaDiCaL) solving time scaling across board sizes N.

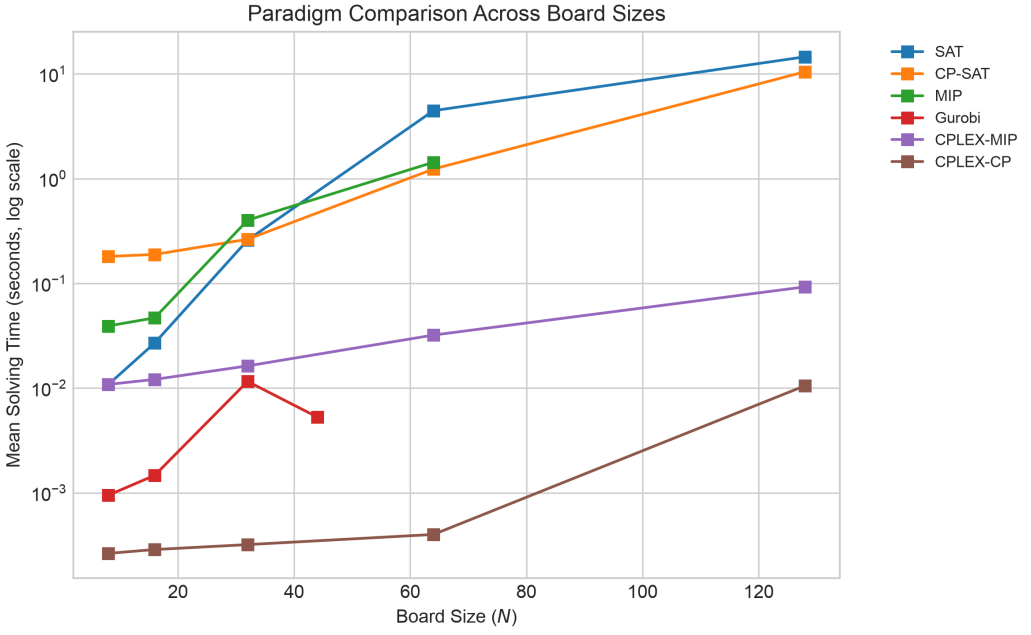


Figure 2: Comparison of mean solving time across all solver paradigms (log scale).

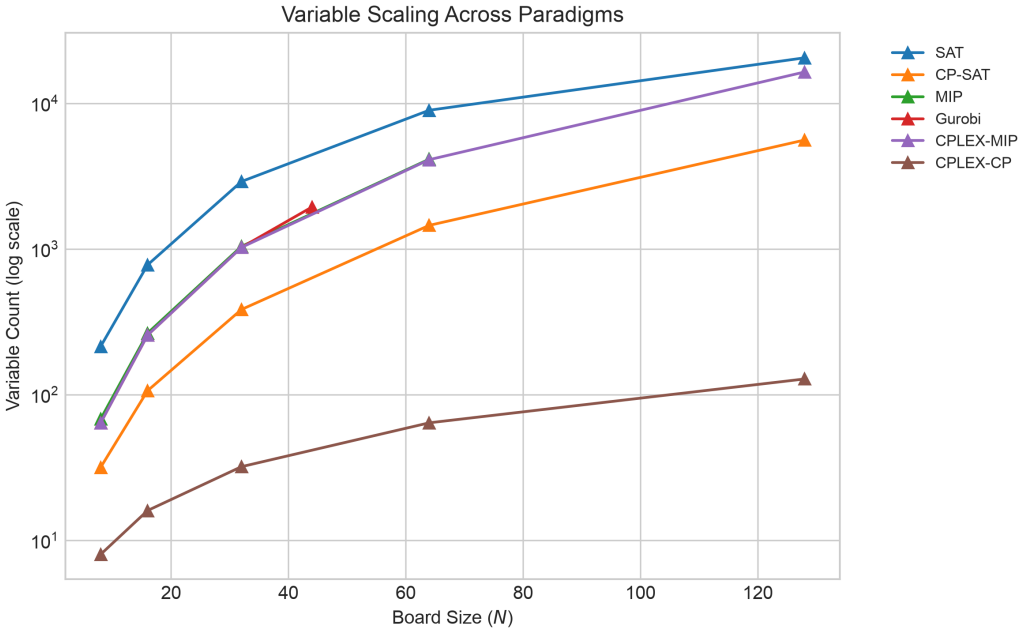


Figure 3: Variable count scaling across different solving paradigms and solvers.

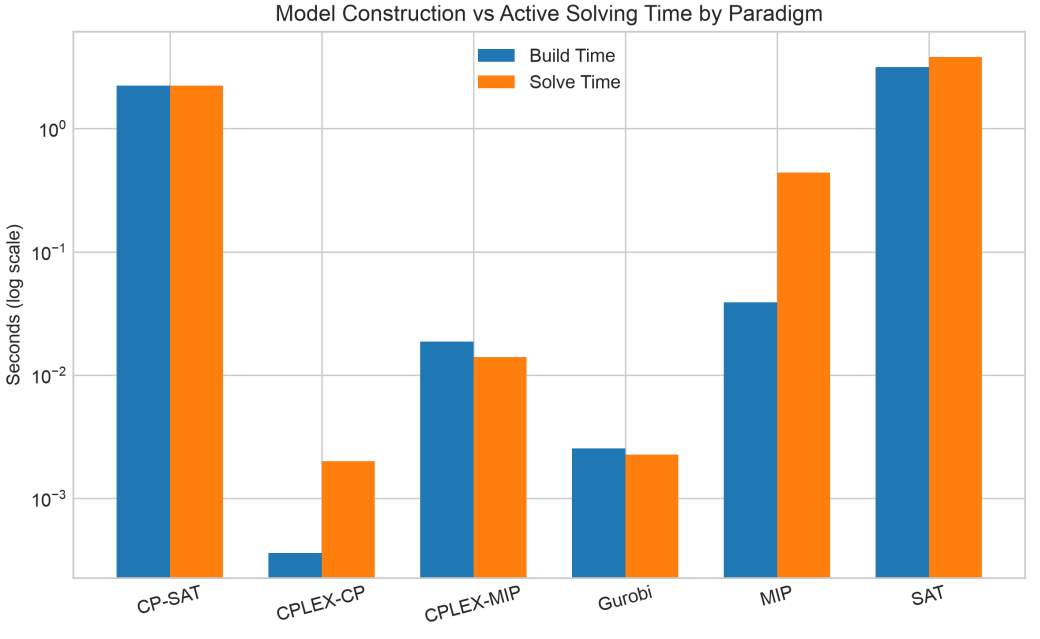


Figure 4: Model construction overhead versus active solving time by paradigm.